ASSIGNMENT

LOW-CODE DEVELOPMENT

Overview and Multiple-Choice Questions

Name: Prasanna K

Roll No.: 240701396

Department: Computer Science and Engineering

5 Topics • 5 Overviews • 100 MCQs

1. The Limits of Low-Code Platforms

Overview

Low-code platforms use visual designers, reusable components, templates, and limited scripting to speed up application development. They make software creation more accessible, but they cannot meet every requirement without technical work. Complex business rules, unusual interfaces, advanced integrations, and high-performance workloads may require custom code or specialist skills. Other concerns include vendor lock-in, licensing costs, feature restrictions, customization limits, migration difficulty, security, privacy, scalability, testing, and long-term maintenance. Teams should evaluate requirements and risks before choosing a platform. Low-code is most effective when its strengths and boundaries are understood early.

Multiple-Choice Questions

1. What is a major purpose of a low-code platform?

A. Eliminate all testing
B. Build apps faster with visual tools and reusable components
C. Replace every database
D. Prevent integrations

2. Which task may require custom code?

A. Change a simple label
B. Select a built-in theme
C. Implement complex unsupported business logic
D. Add a standard text field

3. What does vendor lock-in mean?

A. An app has no users
B. Moving away from a vendor becomes difficult or costly
C. A developer forgets a password
D. A database is offline

4. What should be considered before choosing a platform?

A. Only its logo
B. Requirements, scale, security, cost, and maintenance
C. Only the number of buttons
D. Whether coding is forbidden

5. Does low-code remove the need for testing?

A. Yes, all apps are correct
B. Only for small apps
C. No, apps still need testing
D. Only when AI is used

6. Which is a possible limitation?

A. Supports every language
B. Limited customization for unusual needs
C. Never has licensing fees
D. Cannot create forms

7. Why can integration be challenging?

A. Systems may use different APIs, formats, and authentication
B. All systems use identical formats
C. Integrations need no credentials
D. Platforms cannot store data

8. What if a platform cannot meet a critical requirement?

A. Ignore it
B. Assess custom code, another service, or another platform
C. Remove security
D. Skip testing

9. Which concern relates to sensitive information?

A. Font size
B. Privacy and access control
C. Button spacing only
D. Screen brightness

10. What if licensing limits are overlooked?

A. App becomes open source
B. Unexpected costs or restrictions may occur
C. Testing is unnecessary
D. App cannot have a title

11. Why evaluate performance?

A. All apps perform identically
B. Workloads and platform limits affect speed and scale
C. It matters only for printing
D. It replaces security testing

12. Which practice improves maintainability?

A. No documentation
B. Clear names, documentation, testing, and controlled changes
C. Hide all logic
D. Avoid reviews

13. What is platform portability?

A. Ability to move or recreate a solution elsewhere
B. Changing a button color
C. A user account
D. A printing method

14. When is low-code especially useful?

A. For every problem without analysis
B. For suitable business apps needing rapid visual development
C. Only for operating systems
D. Only when requirements do not exist

15. What can ungoverned citizen development cause?

A. Consistent documentation always
B. Duplicate apps, inconsistent data, and security risks
C. Guaranteed lower cost
D. No maintenance

16. What should be reviewed for a mission-critical app?

A. Only its icon
B. Reliability, security, performance, recovery, and support
C. Only number of pages
D. Nothing if built visually

17. Why might migration be difficult?

A. All platforms have identical data models
B. Features and logic may not map directly
C. Migration never involves data
D. The app has few users

18. What does scalability describe?

A. Ability to handle growth in users or workload
B. Interface color
C. Code comment count
D. Project name length

19. Which statement is accurate?

A. Always cheapest
B. Removes all technical risks
C. Speeds suitable development but has limits
D. Guarantees perfect security

20. How should limitations be managed?

A. Identify early and assess trade-offs
B. Wait until deployment
C. Ignore them
D. Disable all integrations

Answer Key

1. B 2. C 3. B 4. B 5. C 6. B 7. A 8. B 9. B 10. B 11. B 12. B 13. A 14. B 15. B 16. B 17. B 18. A 19. C 20. A

2. Glimpses of Pro Code in Zoho Creator and Catalyst

Overview

Pro-code development uses programming to implement software behavior and control details. Zoho Creator combines visual application building with Deluge scripting, workflows, forms, reports, and integrations. Deluge can implement business rules and automate actions beyond basic drag-and-drop configuration. Zoho Catalyst is a developer platform for building and deploying application components and backend services, including code-based logic and integrations. Creator is often suited to rapidly building business applications, while Catalyst can support more code-centric services. The right choice depends on requirements, team skills, architecture, and the current product capabilities described in official documentation.

Multiple-Choice Questions

1. What is Deluge?

A. Unrelated database brand
B. Scripting language used to add logic in Zoho products
C. Operating system
D. Design font

2. What does pro-code development involve?

A. Only drag-and-drop
B. Writing code to implement software behavior
C. Avoiding APIs
D. Spreadsheets only

3. Which product is known for visual business app building?

A. Zoho Creator
B. Zoho Writer only
C. Zoho Mail only
D. Zoho Meeting only

4. What is scripting commonly used for in Creator?

A. Custom business rules and workflow actions
B. Replacing every account
C. Changing operating systems
D. Disabling forms

5. What is an API used for?

A. Connecting systems through defined interfaces
B. Formatting paper
C. Increasing monitor resolution
D. Replacing requirements

6. Which best describes Zoho Catalyst?

A. Developer platform for application components and backend services
B. Word processor only
C. Network cable
D. Printer type

7. Which capability is associated with code-centric development?

A. Custom functions and service logic
B. Only selecting templates
C. Only moving icons
D. Avoiding data

8. Why choose between Creator and Catalyst?

A. They are identical in every case
B. Needs may call for visual business apps, custom services, or both
C. Neither supports development
D. To avoid testing

9. What should be checked before choosing a Catalyst service?

A. Current official documentation
B. Only social media comments
C. Laptop wallpaper
D. Color palette

10. Which skill helps integration?

A. APIs, data formats, authentication
B. Keyboard shortcuts only
C. Avoiding errors
D. Ignoring networks

11. What is a workflow?

A. Sequence of actions triggered by an event or condition
B. Computer shape
C. Font family
D. Monitor setting

12. Why is error handling important?

A. Makes bugs impossible
B. Helps respond safely when operations fail
C. Removes users
D. Prevents logging

13. Which is a sensible development sequence?

A. Code and never test
B. Understand, design, implement, test, deploy
C. Deploy before understanding
D. Avoid documentation

14. Authentication concerns what?

A. Verifying identity
B. Chart color
C. Image compression only
D. Alphabetical file sorting

15. Authorization concerns what?

A. What an authenticated user may do
B. Screen size
C. Project title
D. Converting data to images

16. Which practice improves code quality?

A. Review and test code
B. Copy unknown code blindly
C. Hard-code public secrets
D. Remove errors

17. Why use a backend function?

A. Run server-side logic or process requests
B. Change keyboard language
C. Print a cover page
D. Replace requirements

18. What is a benefit of visual tools plus code?

A. Rapid development and customization
B. No maintenance ever
C. No security needed
D. Automatic compatibility with all systems

19. What should not be committed to a public repository?

A. Public README
B. Passwords, private keys, secret API credentials
C. Generic function name
D. License notice

20. Best source for current product capabilities?

A. Official documentation
B. Old unverified screenshot only
C. Guess
D. Rumor

Answer Key

1. B 2. B 3. A 4. A 5. A 6. A 7. A 8. B 9. A 10. A 11. A 12. B 13. B 14. A 15. A 16. A 17. A 18. A 19. B 20. A

3. The No-Code to Low-Code to Pro-Code Spectrum Revisited

Overview

No-code emphasizes visual configuration and predefined components with little or no programming. Low-code also uses visual tools but permits scripting or custom code for complex logic and integrations. Pro-code relies primarily on conventional programming and offers detailed control over architecture and implementation. These are not completely separate worlds: a solution may use a no-code prototype, low-code workflows, and custom services where needed. Teams should select an approach based on complexity, customization, integrations, security, performance, budget, skills, and maintenance. Not every low-code application must eventually be rewritten in pro-code.

Multiple-Choice Questions

1. What does no-code emphasize?

A. Visual configuration with little or no programming
B. Kernel drivers
C. Machine-code entry
D. Avoiding templates

2. What best describes low-code?

A. Visual tools plus optional scripting or code
B. Only assembly
C. No configuration
D. Hardware repair

3. What best describes pro-code?

A. Conventional programming with detailed control
B. Only filling forms
C. Only selecting themes
D. Avoiding algorithms

4. Which generally offers most implementation control?

A. No-code
B. Pro-code
C. Static template
D. Presentation tool

5. Can approaches be combined in one solution?

A. No, incompatible
B. Yes, if justified by requirements
C. Only without data
D. Only for games

6. What should guide the approach?

A. Complexity, customization, integration, security, skills, cost
B. Developer name only
C. Tool color
D. Always pro-code

7. What may suit a simple internal request form?

A. No-code or low-code, depending on needs
B. Always OS development
C. Machine code only
D. No development

8. What may favor pro-code?

A. Specialized algorithms and fine-grained control
B. Simple static label
C. Basic form in every case
D. No users

9. Must every app move from no-code to pro-code?

A. Yes, all apps must be rewritten
B. No, a suitable app can remain on its platform
C. Yes, after a month
D. Only if titled

10. What is a prototype?

A. Early version to explore or validate an idea
B. Final security audit
C. Password
D. Backup schedule

11. What is a trade-off?

A. Gaining benefits while accepting costs or limitations
B. Guarantee nothing is lost
C. Avoiding decisions
D. Variable type

12. Which shows low-code flexibility?

A. Visual workflow plus script for a special rule
B. Prohibit every custom rule
C. Paper form only
D. Disable integrations

13. What is abstraction?

A. Hiding lower-level details behind simpler tools
B. Removing all functionality
C. Showing every machine instruction
D. Deleting database

14. Why use different approaches for different components?

A. Components may have different complexity and control needs
B. Every component must use one tool
C. It prevents integration by definition
D. It eliminates architecture

15. Which generally requires stronger programming knowledge?

A. Pro-code
B. Basic no-code configuration
C. Choosing a template
D. Dragging a field

16. Risk of choosing a tool without requirements analysis?

A. It may not support important needs
B. Guaranteed success
C. Testing complete
D. Costs cannot change

17. How validate an approach early?

A. Build a proof of concept and test critical needs
B. Wait until the end
C. Avoid users
D. Skip integration checks

18. What is interoperability?

A. Systems working or exchanging information together
B. Text field size
C. Dashboard color
D. Developer count

19. Which spectrum statement is correct?

A. A range of overlapping approaches
B. A strict ladder everyone must climb
C. No-code is always insecure
D. Pro-code never uses libraries

20. Central lesson of the spectrum?

A. Choose the simplest approach that meets needs and preserves control
B. Always choose newest tool
C. Avoid documentation
D. Code even when useless

Answer Key

1. A 2. A 3. A 4. B 5. B 6. A 7. A 8. A 9. B 10. A 11. A 12. A 13. A 14. A 15. A 16. A 17. A 18. A 19. A 20. A

4. Career Pathways

Overview

Low-code ecosystems offer roles such as citizen developer, low-code application developer, business analyst, automation specialist, application consultant, solution architect, integration developer, QA tester, and platform administrator. Work may include requirements gathering, designing forms and reports, creating workflows, writing scripts, connecting APIs, testing applications, managing access, and supporting users. Learners should start with problem-solving, data fundamentals, and one platform, then build practical projects and learn scripting, APIs, testing, security, databases, cloud concepts, and version control. A portfolio demonstrating real problems and working solutions can help show practical ability. Continuous learning is important as platforms and practices evolve.

Multiple-Choice Questions

1. Who often gathers and clarifies business requirements?

A. Business analyst
B. Network cable
C. Printer
D. Database row

2. What does a low-code developer do?

A. Builds and maintains apps using visual tools and logic
B. Repairs monitors only
C. Writes slogans only
D. Avoids requirements

3. What is a citizen developer?

A. Business user who builds apps using approved tools, often with limited coding
B. National database
C. Cloud server brand
D. Software license

4. Who designs the overall technical solution?

A. Solution architect
B. End user only
C. Printer operator
D. Data field

5. What does an automation specialist focus on?

A. Automating repeatable business processes
B. Replacing policies
C. Avoiding analysis
D. Disabling notifications

6. Which skill is broadly useful?

A. Problem-solving and requirements analysis
B. Avoid communication
C. Ignore testing
D. Memorize colors

7. Why is a project portfolio valuable?

A. Demonstrates practical skills and completed work
B. Replaces all knowledge
C. Guarantees a job
D. Removes interviews

8. What matters for integration roles?

A. APIs and data formats
B. Image filters only
C. Keyboard hardware
D. Video editing only

9. What is the aim of quality assurance?

A. Check requirements and identify defects
B. Create all passwords
C. Avoid test cases
D. Remove documentation

10. Why are communication skills important?

A. Understand stakeholders and explain solutions
B. Replace all technical skills
C. Prevent every defect
D. Eliminate planning

11. Which foundation helps with business data?

A. Database and data-model concepts
B. Animation only
C. Printer maintenance
D. Audio mixing

12. What is a useful first project?

A. Small application solving a clear real-world problem
B. Huge system without requirements
C. Copied project not understood
D. Untested application

13. Why include access control in learning?

A. Business apps may handle restricted data
B. It changes colors only
C. It is unnecessary
D. It replaces authentication

14. Which skills support advanced roles?

A. Programming, databases, cloud, software design
B. Avoid source control
C. Ignore APIs
D. Default settings forever

15. What does a platform administrator manage?

A. Configuration, users, access, governance
B. Advertisements only
C. Weather
D. Personal phone contacts

16. Why is continuous learning important?

A. Platforms and practices evolve
B. Skills never change
C. Features never appear
D. Documentation is irrelevant

17. What demonstrates workflow skill?

A. Build and test an approval process with conditions
B. Change wallpaper
C. Delete requirements
D. Ignore exceptions

18. How prepare for an interview?

A. Explain goals, design choices, challenges, tests, and results
B. Memorize title only
C. Hide mistakes
D. Avoid decisions

19. Why is documentation useful?

A. Helps others understand and maintain a solution
B. Makes testing impossible
C. Removes collaboration
D. Is decorative only

20. What builds a strong career foundation?

A. Platform skills, problem-solving, communication, testing, security
B. Certificate without practice
C. Definitions only
D. Avoid feedback

Answer Key

1. A 2. A 3. A 4. A 5. A 6. A 7. A 8. A 9. A 10. A 11. A 12. A 13. A 14. A 15. A 16. A 17. A 18. A 19. A 20. A

5. The Future of Low-Code

Overview

Low-code will likely continue evolving as organizations seek faster ways to digitize processes. Visual development, reusable components, automation, cloud services, and AI-assisted tools may speed up application creation. AI can suggest workflows, draft code, explain errors, and help create prototypes, but results still need human review and testing. Future adoption also depends on governance, security, privacy, integration, accessibility, performance, and maintainability. Organizations need rules for application ownership, data protection, change review, and avoiding duplicated solutions. Low-code is unlikely to remove professional developers; instead, it can reduce repetitive work and allow more focus on architecture, complex logic, and quality.

Multiple-Choice Questions

1. What drives low-code adoption?

A. Demand for faster digital application delivery
B. Ban on software
C. End of cloud services
D. Need to remove data

2. How may AI assist low-code?

A. Suggest workflows, draft code, explain errors
B. Guarantee error-free apps
C. Remove review
D. Decide every policy

3. Why review AI-generated code?

A. It may be incorrect, insecure, or unsuitable
B. It is always malicious
C. It never runs
D. Review always slows apps

4. What is governance?

A. Rules for building, securing, managing, maintaining apps
B. Choosing a logo
C. Programming language
D. Screen resolution

5. What matters when more employees build apps?

A. Security, access, standards, ownership
B. Font only
C. No documentation
D. No reviews

6. What is a reusable component?

A. Building block used in multiple apps or processes
B. One-time error
C. Keyboard key
D. Password

7. Why will professional developers remain needed?

A. Architecture, custom logic, security, integration, and quality require expertise
B. Platforms cannot display forms
C. All apps need assembly
D. Developers only print

8. Benefit of AI-assisted prototyping?

A. Speeds exploration and initial implementations
B. Guarantees success
C. Removes feedback
D. Prevents changes

9. What is shadow IT?

A. Technology used or built outside approved oversight
B. Dark theme
C. Cloud backup
D. Testing framework

10. How reduce duplicated or risky apps?

A. Governance, approved tools, documentation, inventory
B. Bypass all review
C. Remove access control
D. Avoid ownership

11. What consider before adopting a new feature?

A. Security, cost, fit, reliability, maintenance
B. Announcement date only
C. Visual design only
D. Nothing if AI

12. How can low-code support digital transformation?

A. Digitize workflows and deliver suitable apps faster
B. Avoid understanding processes
C. Prevent data exchange
D. Replace all employees

13. Responsible use of AI-generated logic?

A. Review, test, and validate against requirements and security
B. Deploy without review
C. Put secrets in prompts
D. Ignore permissions

14. Why does data privacy remain important?

A. Apps may process personal, business, or sensitive data
B. Apps never store data
C. Privacy only applies to paper
D. AI grants consent

15. What may become more common in low-code tools?

A. AI assistance and natural-language configuration
B. Machine-code entry
C. Removing integrations
D. Eliminating interfaces

16. What is accessibility?

A. Designing so people with different abilities can use an app
B. Restricting one screen size
C. Using color alone
D. Removing keyboard support

17. Why is lifecycle management important?

A. Apps need controlled development, deployment, updates, retirement
B. Apps never change
C. No owners needed
D. Only paper forms

18. Which future statement is balanced?

A. Productivity may rise, but governance and judgment remain necessary
B. Replace all developers certainly
C. Remove every risk
D. Make testing obsolete

19. Benefit of cloud development platforms?

A. Managed services and deployment options, depending on platform
B. Guaranteed offline operation
C. No permissions needed
D. Compatibility with every system

20. How keep apps sustainable?

A. Review architecture, test changes, document, monitor, maintain
B. Stop updating
C. Avoid performance measures
D. Remove feedback

Answer Key

1. A 2. A 3. A 4. A 5. A 6. A 7. A 8. A 9. A 10. A 11. A 12. A 13. A 14. A 15. A 16. A 17. A 18. A 19. A 20. A